

LAB REPORT 11

STRINGS IN C++

1. OBJECTIVE

- To understand the concept of strings as collections of characters.
- To differentiate between C-Strings (character arrays) and C++ string objects.
- To learn how to declare and initialize C-Strings.
- To utilize built-in functions for string manipulation (e.g., strcpy, strcat, strlen).

2. THEORETICAL BACKGROUND

C-String. In C programming, the collection of characters is stored in the form of arrays. This is also supported in C++ programming. That is why it is called C-strings. C-strings are arrays of type char terminated with a null character, that is, '\0' (ASCII value of null character is 0).

Declaring and Initializing. A C-String can be declared and initialized by providing a string literal, such as `char myString[] = "Hello";`. Although "Hello" has 5 characters, a null character is automatically added to the end. Unlike arrays, it is not necessary to use all the space allocated for the string.

String Object. In C++, you can also create a string object for holding strings. Unlike using char arrays, string objects have no fixed length, and can be extended as per your requirement. Using the standard C++ library string class provides a more robust and flexible way to handle text.

Reading Strings. When reading strings with spaces, the standard `cin >>` operator stops at the first space. To read an entire line of text including spaces, the `cin.get()` function is used for char arrays, and the `getline()` function is used for string objects.

3. SOFTWARE USED

- IDE: DEV-C++ IDE
- Compiler: C++ Compiler (bundled with DEV-C++)

4. EXPERIMENTAL TASKS

Task 1: String Length Calculation

A C++ program was created to declare and initialize a string variable with a specific phrase. The built-in length calculation function was then utilized to determine the number of characters in the string, which was subsequently displayed to the console.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    // Declare and initialize a string variable
    string message = "Welcome to the Electrical Engineering Department.";

    // Calculate and display the length of the string
    cout << "The message is: " << message << endl;
    cout << "The length of the string is: " << message.length() << "
characters." << endl;

    return 0;
}
```

Task 2: String Concatenation

Two separate string variables were declared and initialized representing a first name and a last name. These strings were then concatenated, with a space inserted between them, to form a complete full name. The final concatenated string was displayed.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    // Declare two string variables
    string firstName = "Qazi";
    string lastName = "Ehsanullah";

    // Concatenate the strings to form a full name
    string fullName = firstName + " " + lastName;

    // Display the full name
    cout << "First Name: " << firstName << endl;
    cout << "Last Name: " << lastName << endl;
    cout << "Full Name: " << fullName << endl;

    return 0;
}
```

5. CONCLUSION

This lab focused on the implementation and manipulation of strings in C++. It highlighted the foundational differences between traditional C-style character arrays and modern C++ string objects. By practicing string initialization, length calculation, and concatenation, students learned to manage and process text-based data effectively. The use of string objects demonstrated a clear advantage in terms of flexibility and ease of use compared to fixed-length character arrays.